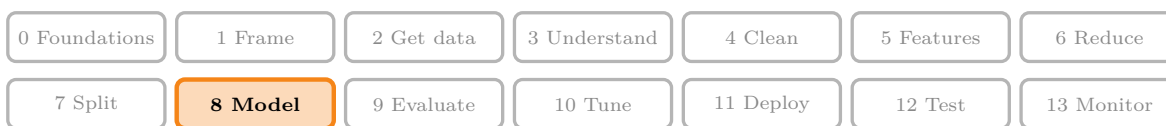


Polynomial Features in Logistic Regression

Contents

1. Overview	2
2. The idea	2
3. An example: two half-moons	3
4. When to use it	4
5. Summary	5
6. Key terms	5

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



1. Overview

Key point: Logistic regression draws a straight boundary. Adding powers and products of the inputs as new columns lets the same algorithm draw curved boundaries, just as polynomial regression did for linear regression.

The perceptron Note stated a limitation: logistic regression works well only when the classes are (almost) linearly separable. On data where the boundary between the classes is curved, a straight line misclassifies many points.

Other algorithms handle such data naturally (decision trees, random forests, SVMs, covered later). But a simple trick also lets logistic regression handle it: **polynomial features**, the same idea as the polynomial regression Note.

2. The idea

Key point: Create new columns x_1^2 , x_1x_2 , x_2^2 , ... and fit ordinary logistic regression on them. A straight boundary in the new columns is a curve in the original ones.

With two inputs x_1 and x_2 , degree 2 creates five columns:

$$x_1, \quad x_2, \quad x_1^2, \quad x_1x_2, \quad x_2^2$$

Logistic regression then learns a weight for each column, so its boundary is

$$w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2 = 0$$

This is the equation of a curve (an ellipse, parabola or hyperbola) in the (x_1, x_2) plane. Higher degrees add more terms (x_1^3 , $x_1^2x_2$, ...) and allow more complicated curves. The output column y is unchanged.

The number of columns grows quickly with the degree: 2 at degree 1, 5 at degree 2, 9 at degree 3, 65 at degree 10 and 350 at degree 25.

Python: Logistic regression on polynomial features.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LogisticRegression
```

```

model = make_pipeline(
    PolynomialFeatures(degree=3, include_bias=False),
    StandardScaler(),
    LogisticRegression())
model.fit(X, y)

```

The scaler puts the power columns on similar scales; without it, high powers such as x^{10} dominate and the solver struggles to converge.

3. An example: two half-moons

Key point: On moon-shaped data, accuracy rises from 0.83 with a straight line to 0.935 with degree 4, then falls slightly at higher degrees.

The data has 200 points in two interlocking half-moon shapes (`make_moons` with noise 0.25). No straight line can separate them. Each model is scored with 10-fold cross-validation (a later Note), which estimates accuracy on unseen data. Regularisation is kept weak ($C=10000$) so that the effect of the degree is visible.



Figure 1: Decision regions for degrees 1, 2, 3, 4, 10 and 25

Degree	Columns	Training accuracy	Cross-validated accuracy
1	2	0.835	0.830
2	5	0.845	0.835

Degree	Columns	Training accuracy	Cross-validated accuracy
3	9	0.960	0.920
4	14	0.955	0.935
10	65	0.955	0.900
25	350	0.955	0.905

- **Degree 1:** plain logistic regression, a straight line. It misclassifies the tips of both moons.
- **Degree 2:** a gentle curve, little better.
- **Degrees 3 and 4:** the boundary bends around the moons. Cross-validated accuracy jumps to 0.92 to 0.935.
- **Degrees 10 and 25:** the boundary develops odd loops and islands far from the data, and cross-validated accuracy drops back to about 0.90.

As with polynomial regression, the degree controls flexibility: too low underfits, too high overfits. It is a hyperparameter, chosen by comparing scores on data not used for training. Here degree 3 or 4 is best.

Extra: With scikit-learn's default regularisation ($C=1$), the high-degree models overfit less, because the penalty keeps the many weights small. This is regularisation doing its job (the Ridge Notes), and it is one more reason to use the default unless there is a reason not to.

4. When to use it

Key point: Useful when only a linear model is available or interpretability matters. For strongly non-linear data, tree-based models or SVMs usually do better.

Polynomial features are a quick way to give logistic regression curved boundaries, and they work well for mildly non-linear data like this example. They have costs:

- the number of columns explodes with many inputs or high degrees, which slows training and invites overfitting;
- the degree must be tuned.

On real data with strongly non-linear patterns, decision trees, random forests or SVMs (later Notes) usually reach better results with less effort.

5. Summary

- Logistic regression's boundary is straight; polynomial features make it curved.
- Degree d adds all powers and products of the inputs up to total power d .
- On the moons data: 0.83 (straight line) rises to 0.935 (degree 4), then falls at higher degrees.
- Choose the degree by cross-validation; standardise the new columns; regularisation limits overfitting.

6. Key terms

Term	Meaning
Polynomial features	New input columns made from powers and products of the original inputs
Decision boundary	The line or curve where the model switches from predicting one class to the other
<code>make_moons</code>	scikit-learn function that creates two interlocking half-moon classes
Cross-validated accuracy	Accuracy averaged over several train-test splits of the data, an estimate of performance on new data